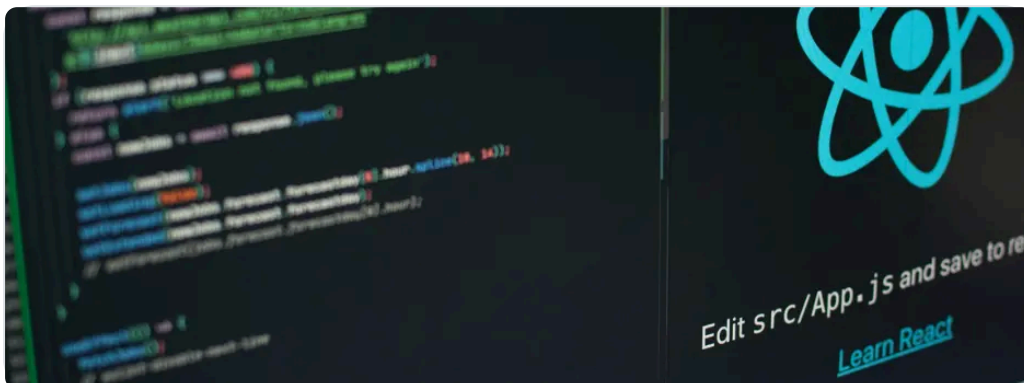




Home > Interview > Top 24 Full Stack Developer Interview Questio...



INTERVIEW

Top 24 Full Stack Developer Interview Questions & Answers

By Jaishree Tomar

Sep 24, 2024 | 14 Min Read | 3988 Views

(Last Updated)



As the field of computer science grows, Full Stack Development has emerged as one of the most popular and in-demand areas.

[Full Stack Developers](#) are skilled individuals who possess expertise in all aspects of web development, including front-end, back-end, and databases. To land a job as a Full Stack Developer, you need to be well-prepared for the interview process.

In this article, we will cover a comprehensive list of Full Stack Developer interview questions and provide detailed answers to help you ace your interviews.

Let's dive into the questions and answers that you can expect during your Full Stack Developer interview.

Table of contents

1. Basic Full Stack Developer Interview Questions

- Mention all the ways by which we can include the CSS code in an HTML code.
- Explain in brief what is meant by the term Semantic Elements in HTML.
- Describe tags, attributes, and elements in HTML.
- What is JavaScript? Explain any four libraries used in JavaScript.
- State the definition of the Callback function in JavaScript, and also give a code snippet as an

Basic Full Stack Developer Interview Questions

In this section, we will cover some entry-level questions that are commonly asked during Full Stack Developer interviews. These questions are relatively easy and can be answered if you have a basic understanding of the topics.

1. Mention all the ways by which we can include the CSS code in an HTML code.

To style an HTML document, we use Cascading Style Sheets (CSS). There are three main ways to include CSS code in an HTML document:

- **Inline Method:** In this method, you can include CSS code directly within the HTML tags using the style attribute. For example:

```
<h1 style="color: blue;">Hello World</h1>
```

- **Internal Method:** You can use the `<style>` tag within the `<head>` section of the HTML document to define CSS styles. For example:

```
<head>
  <style>
    h1 { color: blue;
  }
</style>
</head>
```

- **External Method:** You can link an external CSS file to your HTML document using the `<link>` tag. The CSS code is written in a separate file with a `.css` extension. For example:

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Also Read: [Top 20 HTML & CSS Interview Questions With Answers](#)

2. Explain in brief what is meant by the term Semantic Elements in HTML.

Semantic Elements in HTML are tags that provide meaning to the structure and content of a web page. These elements represent the different sections and layouts of a web page in a more readable and understandable manner.

Semantic elements not only describe the appearance of the content but also convey its meaning to both the browser and the developer.

Examples of semantic elements in HTML include `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>`, `<footer>`, etc. These elements help search engines and screen readers understand the structure and purpose of the content, resulting in better accessibility and SEO.

Explore More: [Introduction to HTML Tags: A Comprehensive Guide With Examples](#)

3. Describe tags, attributes, and elements in HTML.

In HTML, tags, attributes, and elements are the building blocks used to create web pages.

- **Tags:** Tags are keywords enclosed in angle brackets (<>) that define HTML elements. They are used to mark up and structure the content of a web page. For example, the <h1> tag is used to define a heading, and the <p> tag is used to define a paragraph.
- **Attributes:** Attributes provide additional information or properties to HTML elements. They are used within the opening tag of an element and consist of a name and a value. For example, the href attribute in the <a> tag specifies the URL of a link, and the src attribute in the tag specifies the source of an image.
- **Elements:** Elements are made up of tags and their content. They represent the structure and semantics of a web page. For example, <h1>Welcome to my website</h1> is an example of an element, where <h1> is the opening tag, and Welcome to my website is the content.

Also Read: [Top Git Interview Questions and Answers For 2024](#)

Elevate Your Career Zen Class
Courses with Placement Guidance

[Learn More](#)



4. What is JavaScript? Explain any four libraries used in JavaScript.

JavaScript is a popular programming language used for both client-side and server-side web development. It is a dynamic, high-level, and interpreted language that allows developers to add interactivity and functionality to web pages.

Here are four commonly used JavaScript libraries:

- **jQuery:** jQuery is a fast, small, and feature-rich JavaScript library. It simplifies HTML document traversal, event handling, animating, and Ajax interactions for rapid web development.

- **React:** React is a JavaScript library for building user interfaces. It allows developers to create reusable UI components and efficiently update and render data-driven views.
- **Angular:** Angular is a TypeScript-based open-source framework for building web applications. It provides a comprehensive set of features for developing scalable and maintainable applications.
- **Vue:** Vue.js is a progressive JavaScript framework for building user interfaces. It focuses on the view layer and can be easily integrated into existing projects.

These libraries provide developers with pre-built functions and tools that make it easier to write complex JavaScript code and enhance the overall development process.

Also Read: [7 Best JavaScript Practices Every Developer Must Follow](#)

5. State the definition of the Callback function in JavaScript, and also give a code snippet as an example.

In JavaScript, a callback function is a function that is passed as an argument to another function and is executed asynchronously. The callback function is called when a specific event or condition occurs.

Here is an example of a callback function in JavaScript:

```
function fetchData(url, callback) {  
  // Simulate an asynchronous request  
  setTimeout(() => {  
    const data = { name: 'GUVI', age: 10 };  
    callback(data);  
  }, 2000);  
}  
  
function displayData(data) {  
  console.log(`Name: ${data.name}, Age: ${data.age}`);  
}
```

```
fetchData('https://api.example.com/data', displayData);
```

In the above example, the `fetchData` function simulates an asynchronous request and takes a callback function as an argument. When the request is complete, the callback function `displayData` is called with the fetched data as the argument.

Callback functions are commonly used in JavaScript for handling asynchronous operations, such as AJAX requests, timers, and event listeners.

6. Expand the terms MEAN stack and MERN stack, and mention their alternatives.

The MEAN stack and MERN stack are two popular stacks used for full-stack web development.

- **MEAN stack:**
 - **MongoDB:** A NoSQL database that stores data in a flexible, JSON-like format.
 - **Express.js:** A web application framework for Node.js that provides a set of features for building web applications and APIs.
 - **Angular:** A front-end JavaScript framework for building user interfaces.
 - **Node.js:** A JavaScript runtime environment that allows server-side execution of JavaScript code.
- **MERN stack:**
 - **MongoDB:** A NoSQL database.
 - **Express.js:** A web application framework for Node.js.
 - **React:** A JavaScript library for building user interfaces.
 - **Node.js:** A JavaScript runtime environment.

Alternatives to the MEAN stack and MERN stack include:

- **LAMP stack:**

- **Linux:** The operating system.
- **Apache:** The web server software.
- **MySQL:** The relational database management system.
- **PHP:** The server-side scripting language.
- **Django stack:**
 - **Django:** A high-level Python web framework.
 - **Python:** The programming language.
 - **SQL:** A relational database management system.

Each stack has its strengths and is suitable for different types of projects and development preferences.

Also Read: [MEAN vs MERN: Career Growth & Salary](#)

7. What is meant by scope in Angular Js? Also, define the different types.

In AngularJS, scope refers to an object that acts as a binding between the view and the controller. It holds the data and methods that are accessible within a particular portion of the DOM (Document Object Model).

There are three types of scope in AngularJS:

- **\$rootScope:** The **\$rootScope** is the top-level scope that is accessible throughout the entire AngularJS application. It is the parent scope for all other scopes and can be used to share data between different controllers.
- **Controller Scope:** Each controller in AngularJS has its scope, which is created when the controller is instantiated. The controller scope is limited to the portion of the DOM controlled by that specific controller.
- **Directive Scope:** Directives in AngularJS can have their isolated scope. This means that the scope of a directive is separate from the controller and other directives. Isolated scopes are useful for creating reusable components.

The scope in AngularJS allows for two-way data binding, where changes in the view are automatically reflected in the scope and vice versa.

8. Briefly explain Directives in Angular Js.

In AngularJS, directives are a way to extend HTML with new behavior and functionality. They allow you to create reusable components and add custom behavior to existing HTML elements.

Directives are defined using the **directive** function and can be used as attributes, elements, classes, or comments in the HTML code. They enhance the functionality of HTML by attaching special behaviors to elements.

Some commonly used directives in AngularJS include:

- **ng-model:** Binds the value of an input element to a property in the scope.
- **ng-repeat:** Iterates over a collection and creates a clone of a template for each item in the collection.
- **ng-show/ng-hide:** Shows or hides an element based on a condition.
- **ng-click:** Binds a function to the click event of an element.

Directives in AngularJS provide a powerful way to create dynamic and interactive web applications.

Find Out: [React vs Angular vs Vue: Choosing the Right Framework](#)

9. What are the features of Node Js? Also, mention why it's a single-threaded framework.

Node.js is a popular JavaScript runtime environment that allows developers to build server-side applications. Here are some key features of Node.js:

- **Asynchronous and Event-Driven:** Node.js uses an event-driven architecture, which allows it to handle multiple requests concurrently without blocking the execution of other code. This makes it highly scalable and efficient.

- **Non-Blocking I/O:** Node.js uses non-blocking, asynchronous I/O operations, which means that it can handle a large number of concurrent connections without consuming excessive system resources.
- **Single-Threaded:** Node.js operates on a single-threaded event loop model. This means that it uses a single thread to handle all incoming requests. However, it employs non-blocking I/O operations and event-driven programming to achieve high concurrency and performance.
- **NPM (Node Package Manager):** Node.js has a vast ecosystem of open-source libraries and packages available through NPM. These packages can be easily installed and used in Node.js applications, making development faster and more efficient.

The single-threaded nature of Node.js allows it to handle a large number of concurrent connections efficiently. By avoiding the overhead of creating and managing multiple threads, Node.js can achieve high scalability and performance.

10. Define and expand the term NPM. Why do we use NPM with Node.js?

NPM (Node Package Manager) is the default package manager for Node.js. It is a command-line tool that allows developers to discover, install, and manage third-party libraries and packages for their Node.js projects.

NPM provides a vast repository of reusable code modules, known as packages, which can be easily installed and used in Node.js applications. It simplifies the process of managing dependencies and ensures that all required packages are installed correctly.

Some key features of NPM include:

- **Package Installation:** NPM allows developers to install packages from the NPM registry or local directories. It automatically resolves and installs any dependencies required by the package.
- **Package Publishing:** Developers can publish their packages to the NPM registry, making them available to the wider community.

- **Version Management:** NPM provides tools for managing package versions, allowing developers to easily update or downgrade packages as needed.
- **Scripts:** NPM allows developers to define custom scripts in the `package.json` file, which can be executed using the `npm run` command. This makes it easy to automate common development tasks.

NPM is an essential tool for Node.js developers as it simplifies package management, promotes code reuse, and helps maintain project dependencies.

Also Read: [6 Essential Prerequisites For Learning ReactJS](#)

Before we move to the next section, make sure that you are strong in the full-stack development basics. If not, consider enrolling for a **professionally [certified online full-stack web development course](#)** by a recognized institution that can also offer you an industry-grade certificate that boosts your resume.

Intermediate Full Stack Developer Interview Questions

In this section, we will cover intermediate-level Full Stack Developer interview questions that are more challenging and require a deeper understanding of Full Stack Development concepts.

1. Explain the concept of RESTful API and its usage.

A RESTful API (Representational State Transfer) is an architectural style that defines a set of constraints for creating web services. It is based on the principles of simplicity, scalability, and statelessness.

A RESTful API allows clients to interact with a server by sending HTTP requests and receiving responses. It operates on the following principles:

- **Resource-based:** A RESTful API treats everything as a resource, which can be accessed and manipulated using standard HTTP methods (GET, POST, PUT, DELETE).

- **Stateless:** The server does not maintain any client state between requests. Each request from the client must contain all the necessary information for the server to process it.
- **Uniform Interface:** A RESTful API has a standardized set of methods (GET, POST, PUT, DELETE) and response codes (200, 201, 400, 404, etc.) that are used consistently across all resources.
- **Cacheable:** Responses from a RESTful API can be cached by the client to improve performance and reduce server load.

RESTful APIs are widely used in web development for creating scalable and interoperable web services. They provide a flexible and efficient way to expose data and functionality to clients.

Must Read: [Conquer Your Next Interview: Top 5 Java Programs for Freshers](#)

2. What do you mean by the term MEAN Stack?

The MEAN stack is a popular full-stack JavaScript framework for building web applications. It is an acronym for the following technologies:

- **MongoDB:** A NoSQL database that stores data in a flexible, JSON-like format.
- **Express.js:** A minimalistic web application framework for Node.js that provides a set of features for building web applications and APIs.
- **Angular:** A front-end JavaScript framework for building dynamic and interactive user interfaces.
- **Node.js:** A JavaScript runtime environment that allows server-side execution of JavaScript code.

The MEAN stack enables developers to build end-to-end JavaScript applications, where both the front-end and back-end are written in JavaScript. It provides a seamless integration between the different components, allowing for efficient development and deployment.

3. What is the purpose of package.json in Node.js?

The `package.json` file is a metadata file used in Node.js projects. It contains various details about the project, such as its name, version, dependencies, scripts, and other configuration information.

Here are some key purposes of the `package.json` file:

- **Dependency Management:** The `package.json` file lists all the dependencies required by the project. When a developer runs `npm install`, NPM reads the `package.json` file and installs all the specified dependencies.
- **Script Execution:** The `package.json` file can define custom scripts that can be executed using the `npm run` command. These scripts can automate common development tasks, such as running tests, starting the server, or building the project.
- **Version Management:** The `package.json` file specifies the version of the project. It allows developers to easily manage and update the version of the project using semantic versioning.
- **Metadata:** The `package.json` file includes various metadata about the project, such as the project name, description, author, license, repository URL, and other relevant information.

The `package.json` file is an essential part of Node.js projects as it provides a centralized location for managing project dependencies and configuration.

4. Explain the concept of asynchronous programming in JavaScript.

- [Asynchronous programming](#) is a programming paradigm that allows multiple tasks to run concurrently without blocking the execution of other code. In JavaScript, asynchronous programming is achieved using callbacks, promises, and `async/await`.
- In traditional synchronous programming, each task is executed one after the other, blocking the execution until the current task is completed. Asynchronous programming, on the other hand, allows tasks to be executed in parallel, improving the overall performance and responsiveness of the application.

- Callbacks are the simplest form of asynchronous programming in JavaScript. A callback function is passed as an argument to another function and is called when a specific event or condition occurs. This allows the application to continue running while waiting for the asynchronous operation to complete.
- Promises provide a more structured approach to asynchronous programming. They represent the eventual completion or failure of an asynchronous operation and allow developers to chain multiple asynchronous operations together.
- Async/await is a syntax introduced in ECMAScript 2017 that allows developers to write asynchronous code in a synchronous manner. It provides a more readable and intuitive way to handle asynchronous operations by using the **async** and **await** keywords.
- Asynchronous programming is crucial in JavaScript as it allows developers to create responsive and efficient applications that can handle multiple tasks concurrently.

5. What is the difference between PUT and POST methods in RESTful APIs?

In RESTful APIs, the PUT and POST methods are used to create or update resources on the server. While they are similar in functionality, there are some key differences between the two methods:

- **POST:** The POST method is used to create a new resource on the server. When a client sends a POST request to a specific endpoint, the server generates a new resource and assigns it a unique identifier. The response from the server typically includes the URL of the newly created resource.
- **PUT:** The PUT method is used to update an existing resource on the server. When a client sends a PUT request to a specific endpoint, the server replaces the current state of the resource with the new state provided in the request payload. If the resource does not exist, the server may create a new resource with the specified identifier.

In summary, the POST method is used to create new resources, while the PUT method is used to update existing resources. The key difference is that the client does not need to know the URL of the resource in advance when using the POST method, whereas the client must specify the URL of the resource to be updated when using the PUT method.

Also Read: [Use ReactJS to Fetch and Display Data from API – 5 Simple Steps](#)

6. What is the purpose of the “use strict” directive in JavaScript?

The “use strict” directive is a feature introduced in ECMAScript 5 (ES5) that enables strict mode in JavaScript. When strict mode is enabled, JavaScript code is executed in a more secure and error-prone manner.

Some key benefits and features of strict mode include:

- **Prevents the use of undeclared variables:** In strict mode, using a variable without declaring it first will throw an error. This helps catch typographical errors and encourages proper variable declaration.
- **Eliminates the use of implicit global variables:** In non-strict mode, assigning a value to an undeclared variable creates a new global variable. Strict mode prevents this behavior, helping to avoid unintended global variable pollution.
- **Disallows duplicate parameter names:** In strict mode, using duplicate parameter names in function declarations or expressions will throw an error. This promotes cleaner and more maintainable code.
- **Prohibits the use of certain JavaScript features:** Strict mode disables or modifies some features that are considered error-prone or potentially problematic. For example, octal literals (e.g., `0123`) are not allowed in strict mode.

To enable strict mode in a JavaScript file, add the following directive at the beginning of the file or within a function:

```
"use strict";
```

Strict mode is recommended for all JavaScript projects as it helps catch common programming mistakes and encourages best practices.

7. What is the purpose of Docker, and what are some of its uses?

Docker is an open-source platform that allows developers to automate the deployment and management of applications in lightweight, isolated containers. Containers are self-contained environments that contain everything needed to run an application, including the code, runtime, system tools, and libraries.

Some of the key uses of Docker include:

- **Application Deployment:** Docker simplifies the deployment of applications by providing a consistent and reproducible environment. Developers can package their applications and dependencies into containers, ensuring that the application runs consistently across different environments.
- **Microservices Architecture:** Docker is well-suited for building and deploying applications based on a microservices architecture. Each microservice can be packaged as a separate container, allowing for independent scaling, deployment, and management.
- **Continuous Integration and Deployment:** Docker integrates well with continuous integration and deployment (CI/CD) pipelines. Developers can use Docker to build, test, and deploy applications in a consistent and automated manner, reducing the risk of configuration errors and ensuring reliable deployments.
- **Development Environments:** Docker can be used to create lightweight, isolated development environments. Developers can set up containers with specific dependencies and configurations, making it easy to share and reproduce development environments across different machines.

- **Scalable Infrastructure:** Docker makes it easy to scale applications by running multiple instances of containers across a cluster of machines. Containers can be easily provisioned and orchestrated using container orchestration tools like Kubernetes.

Docker has revolutionized the way applications are deployed and managed, providing developers with a powerful and flexible platform for building and running applications.

8. Explain the 'event' loop in Node.js.

In Node.js, the event loop is the core mechanism that allows non-blocking, asynchronous I/O operations to be handled efficiently. It is responsible for processing events and executing callbacks in an event-driven architecture.

The event loop works as follows:

1. **Event Registration:** Asynchronous operations, such as reading from a file or making an HTTP request, are registered with the event loop. The event loop keeps track of these operations and their associated callbacks.
2. **Event Queue:** When an asynchronous operation is completed, its callback is placed in the event queue. The event queue holds all the completed callbacks, waiting to be processed.
3. **Event Loop Execution:** The event loop continuously checks for completed callbacks in the event queue. If there are any callbacks, it picks one and executes it. This process is repeated until the event queue is empty.
4. **Blocking Operations:** If a blocking operation, such as a CPU-intensive computation, is encountered during the event loop execution, it is offloaded to a separate thread or process. This ensures that the event loop remains responsive and can handle other events and callbacks.

The event loop in Node.js allows for high concurrency and scalability by efficiently handling multiple asynchronous operations concurrently. It enables developers to write non-blocking, asynchronous code that can handle a large number of concurrent connections without blocking the execution of other code.

9. Is there a way to decrease the load time of a web application?

Yes, there are several techniques to decrease the load time of a web application and improve its overall performance:

- **Optimize and Minify Code:** Minifying and compressing JavaScript, CSS, and HTML files can significantly reduce their file size and improve load times. Removing unnecessary whitespace, comments, and code can also help optimize the performance of the application.
- **Enable Browser Caching:** By setting appropriate cache headers, you can instruct the browser to cache static assets like images, CSS, and JavaScript files. This allows the browser to store these assets locally, reducing the number of requests required to load the page.
- **Optimize Images:** Images often contribute to the largest file size of a web page. Optimizing and compressing images using tools like ImageMagick or online services can help reduce their size without compromising quality.
- **Lazy Loading:** Lazy loading is a technique that defers the loading of non-critical resources, such as images or videos, until they are needed. This can significantly improve the initial load time of a page by reducing the number of requests and the overall page size.
- **CDN (Content Delivery Network):** Using a CDN can help distribute static assets across multiple servers located in different geographical locations. This reduces the distance between the user and the server, resulting in faster load times.
- **Gzip Compression:** Enabling Gzip compression on the server can significantly reduce the size of the transferred data. Gzip compresses the response before sending it to the client, reducing the time and bandwidth required to load the page.
- **Optimize Database Queries:** Database queries can be a bottleneck in web applications. Optimizing queries, indexing the

database, and caching frequently accessed data can help improve the performance of database operations.

- **Reduce HTTP Requests:** Minimizing the number of HTTP requests can significantly improve load times. Combining multiple CSS or JavaScript files into a single file, using CSS sprites for images, and reducing the number of external resources can help reduce the number of requests made by the browser.

By implementing these optimization techniques, you can decrease the load time of your web application and provide a better user experience.

***Also Explore:** [How To Prepare for Any Job Interview? | 7 Best Hacks](#)*

Advanced Full Stack Developer Interview Questions

In this section, we will cover advanced-level Full Stack Developer interview questions that require a deeper understanding of Full Stack Development concepts and technologies.

1. State the difference between blue/green deployment and rolling deployment.

Blue/green deployment and rolling deployment are two strategies used for deploying new versions of an application while minimizing downtime and risk.

- **Blue/Green Deployment:** In a blue/green deployment, two identical environments (blue and green) are set up, with only one environment serving live traffic at a time. The blue environment represents the current live version of the application, while the green environment represents the new version.

The deployment process involves the following steps:

1. The new version of the application is deployed to the green environment.
2. The green environment is thoroughly tested and validated.

3. Once the green environment is deemed stable and ready, a traffic switch is performed, directing all incoming traffic to the green environment.
 4. The blue environment is now idle and can be used for future deployments or rolled back to in case of any issues with the green environment.
- **Rolling Deployment:** In a rolling deployment, new versions of the application are deployed incrementally to a subset of servers or instances while the remaining servers continue to serve live traffic. The deployment process involves the following steps:
 1. The new version of the application is deployed to a small subset of servers or instances.
 2. The newly deployed servers are tested and monitored to ensure they are functioning correctly.
 3. If the newly deployed servers pass the tests, the deployment continues to the next subset of servers.
 4. This process is repeated until all servers have been updated with the new version.

The key difference between blue/green deployment and rolling deployment is that blue/green deployment involves switching the entire traffic from one environment to another while rolling deployment involves deploying the new version incrementally without interrupting the live traffic.

2. Explain the concept of inversion of control.

Inversion of Control (IoC) is a design principle that specifies the inversion of traditional control flow in a software application. In traditional programming, the application controls the flow of execution by invoking methods and managing dependencies. In contrast, IoC transfers the control of managing dependencies and executing methods to a container or framework.

The main idea behind IoC is to decouple components and promote loose coupling and modular design. Instead of hardcoding

dependencies within a component, IoC allows the dependencies to be injected or provided by an external entity.

IoC can be achieved using techniques such as dependency injection (DI) and the service locator pattern. Dependency injection is a popular approach that involves passing dependencies as arguments to a component's constructor or through setters.

By adopting IoC, applications become more flexible, modular, and easier to test. It allows for the easy replacement of dependencies, promotes code reuse, and simplifies the management of complex systems.

3. What do you mean by the term CI/CD (Continuous Integration/Continuous Deployment)?

CI/CD stands for Continuous Integration/Continuous Deployment, which are practices aimed at improving the software development and deployment process.

- **Continuous Integration (CI):** CI is the practice of frequently merging code changes from multiple developers into a shared repository. Each code change triggers an automated build and test process to ensure that the changes integrate smoothly with the existing codebase and do not introduce any regressions or conflicts.

CI helps catch integration issues early on, encourages collaboration among team members, and ensures that the codebase is always in a releasable state. Common CI tools include Jenkins, Travis CI, and CircleCI.

- **Continuous Deployment (CD):** CD is the practice of automatically deploying application changes to production as soon as they pass the CI process. This ensures that new features, bug fixes, and enhancements are delivered to end-users quickly and efficiently.

CD involves automating the deployment process, including tasks such as building the application, running tests, and deploying to production servers. By automating the deployment process, CD reduces the risk of human error, saves time, and enables faster feedback loops.

Together, CI/CD practices promote a culture of frequent code integration, automated testing, and continuous delivery, resulting in higher quality software, faster time to market, and improved development efficiency.

Know More: [Continuous Integration vs Continuous Deployment in DevOps \(A Comprehensive Guide\)](#)

4. State the difference between normalization and denormalization.

Normalization and denormalization are techniques used in database design to optimize data storage and query performance.

- **Normalization:** Normalization is the process of organizing data in a database to eliminate redundancy and dependency anomalies. It involves breaking down a database into smaller, well-structured tables that store atomic and non-redundant data.

Normalization follows a set of rules called Normal Forms (NF), such as First Normal Form (1NF), Second Normal Form (2NF), and so on. By normalizing data, we ensure data integrity, minimize data redundancy, and make the database more flexible and scalable.

- **Denormalization:** Denormalization is the process of combining normalized tables to improve query performance by reducing the number of joins required. It involves duplicating data from related tables into a single table, trading off redundancy for improved query performance.

Denormalization is often used in read-intensive applications, where query performance is critical. By denormalizing data, we can reduce the number of joins required to retrieve data, resulting in faster query execution.

The choice between normalization and denormalization depends on the specific requirements of the application. Normalization is recommended for data integrity and flexibility, while denormalization is suitable for improving query performance in read-intensive scenarios.

5. In Java, what is a connection leak? How can you fix this?

In Java, a connection leak occurs when a database connection is not properly closed after it has been used. This can happen if an exception is thrown before the connection is closed, or if the connection is not explicitly closed in the code.

Connection leaks can lead to resource exhaustion, as each unclosed connection consumes system resources, such as memory and network sockets. Over time, this can result in degraded application performance and even application crashes.

To fix connection leaks, it is important to ensure that connections are always properly closed when they are no longer needed. There are several approaches to achieve this:

- **Using try-with-resources:** In Java 7 and later, you can use the try-with-resources statement to automatically close resources, such as database connections, at the end of a block, even if an exception occurs. This ensures that connections are always closed, even in the presence of exceptions.

```
try (Connection connection = dataSource.getConnection()) {  
    // Use the connection  
} catch (SQLException e) {  
    // Handle the exception  
}
```

Using a final block: If you are using an older version of Java, you can manually close the connection in a **finally** block, which is guaranteed to execute, regardless of whether an exception occurs.

```
Connection connection = null;  
try {  
    connection = dataSource.getConnection();  
    // Use the connection  
} catch (SQLException e) {  
    // Handle the exception  
} finally {  
    if (connection != null) {  
        try {  
            connection.close();  
        } catch (SQLException e) {  

```

```
        // Handle the exception
    }
}
}
```

Must Read: [Top 100 Core Java Interview Questions and Answers](#)

If you want to learn more about Frontend Development and make a successful career out of it, then you must sign up for the [Certified Full Stack Development Course](#), offered by GUVI, which gives you in-depth knowledge of the practical implementation of all frontend as well as backend development through various real-life FSD projects.

Takeaways...

An employer does not expect candidates to possess all-encompassing knowledge, whether they are beginners or seasoned professionals. However, a well-prepared and enthusiastic applicant will undeniably stand out.

During a full stack developer interview, candidates should be capable of showcasing exceptional organizational skills and accuracy in their work. Additionally, honesty and transparency prove to be advantageous in the long term.

This article encompasses the top 24 interview questions and answers for freshers and experienced candidates in the field of Full-Stack development.

By familiarizing yourself with these questions, you will be able to excel in your next full-stack developer interview. It is recommended that you thoroughly go through all the questions and answers provided.

We hope that we have addressed any uncertainties you may have had and have steered you in the right direction. Best of luck with your full stack developer interview!

Also Explore: [Top Full Stack Development Trends: What to Expect](#)

What are the basics of full stack developer?



What full stack means?



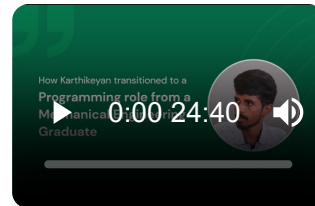
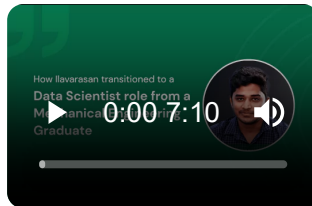
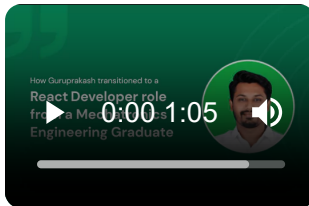
What are the types of full stack?



Why hire full stack developer?



Career transition



About the Author

Jaishree Tomar

A recent CS Graduate with a quirk for writing and coding, a Data Science and Machine Learning enthusiast trying to pave my own way with tech. I have worked as a freelancer with a UK-based Digital Marketing firm writing various tech blogs, articles, and code snippets. Now, working as a Technical Writer at GUVI writing to my heart's content!

Connect with me @



Did you enjoy this article?

Name

Email

Rate this Article     

Enter Your Comment

Submit My Comment

Career Building Program in **Full Stack Development**



Interview Coaching



24 Hrs AI Chat Support



Portfolio Building



Communication Skills





Join the World's Largest Women-Only Gen AI Learning Challenge

 Sept 21st, 2024

Exclusive Access & Celebrations

Learnathon & Hackathon

Courses & Certifications

[Register Now](#)



Get In Touch For Details!

Name*

Email ID*

Phone Number*

Select your interested program*

Get In Touch Request Information

Recommended Courses

Career Programs

Micro Courses



MOST POPULAR

Java Full Stack Development Course

Available in

ENGLISH TAMIL

Know More >>

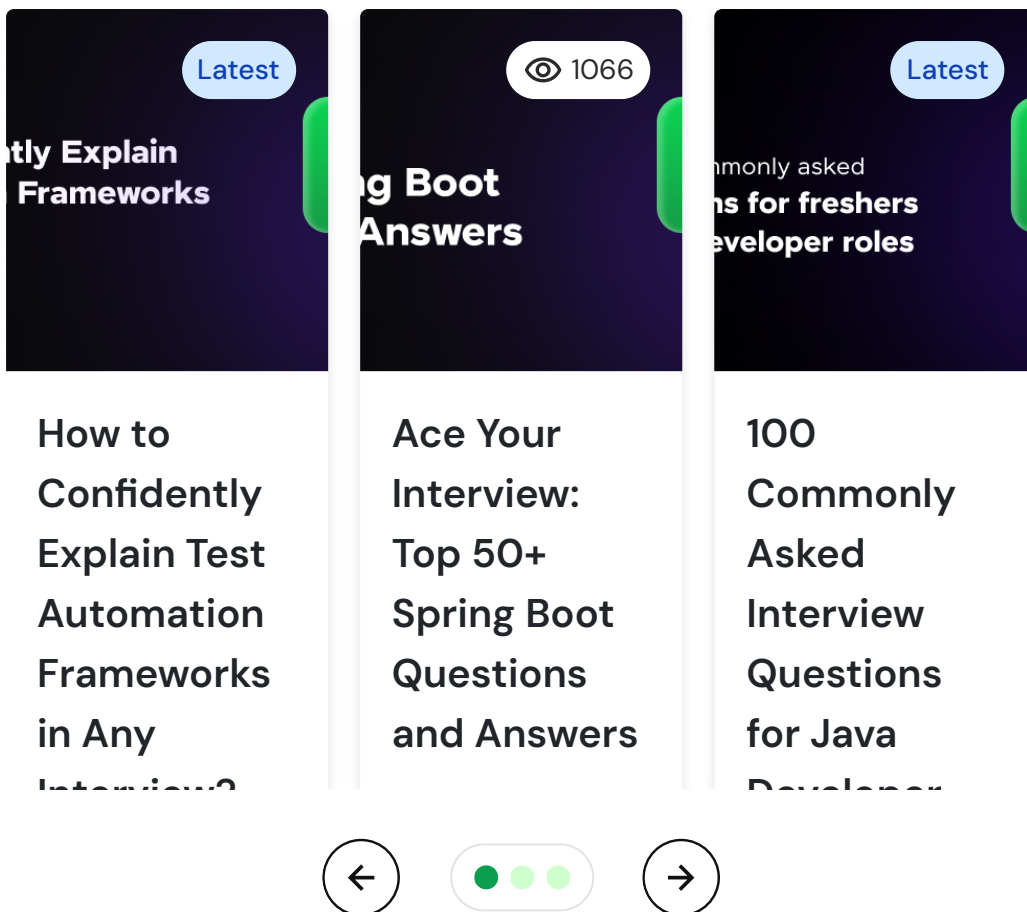
GUVI H.S.D.E. Full Stack Development Course
Transforming the Skill Landscape



Schedule 1:1 free counselling

Talk to Career Expert

Similar Articles



Blog Categories



Data Science

Artificial Intelligence and Machine Learning

Blockchain

Cloud Computing

Cyber Security

Digital Marketing

Full Stack Development

Interview Questions



General Interview Questions

Java Interview Questions

[Python Interview Questions](#)

[SQL Interview Questions](#)

[Selenium Interview Questions](#)

[Data Science Interview Questions](#)

Salary blog



[UI/UX Designer Salary](#)

[Data Scientist Salary](#)

[Full-Stack Developer Salary](#)

[Motion Graphics Designer Salary](#)

[Cloud Computing Engineer Salary](#)

[Digital Marketer Salary](#)

About us



[Our Story](#)

[Careers](#)

[Refund Policy](#)

[FAQs](#)

[Contact Us](#)



[Refer & Earn](#)

GUVI – India's Pioneering Vernacular EdTech, incubated by IIT-M, IIM-A, and a prestigious part of the HCL group. Empowering over 2.5 million global learners with top-tier educational solutions through a vernacular approach, we have revolutionized global tech education with strategic partnerships, including 'Google for Education,' AICTE,

AWS, Google Cloud, IIT-M Pravartak, UiPath, and NASSCOM. GUVI is your trusted guide to inclusive and accredited learning experiences.



Follow us on



[Terms and Conditions](#)

[Privacy Policy](#)

© GUVI Geeks Network Pvt. Ltd

 [Table of contents](#)